

Highlights

MooseAgent: A LLM Based Multi-agent System for Automating Moose Simulation

Tao Zhang, Zhenhai Liu, Guangliang Yang, Yong Xin, Yuanming Li, Yongjun Jiao

- MooseAgent: A LLM Based Multi-agent System for Automating Moose Simulation.
- Using a data-model fusion driven method to train the network to solve the problem of sparse data.
- Achieved a 25% improvement in accuracy compared to classical semi-empirical model.

MooseAgent: A LLM Based Multi-agent System for Automating Moose Simulation

Tao Zhang^{a,*}, Zhenhai Liu^a, Guangliang Yang^a, Yong Xin^a, Yuanming Li^a and Yongjun Jiao^{a,**}

^aState Key Laboratory of Advanced Nuclear Energy Technology, Nuclear Power Institute of China, Chengdu, 610213, China

ARTICLE INFO

Keywords:

Large language model
Multi agent system
Moose
Automated simulation

ABSTRACT

The Finite Element Method (FEM) is widely applied in engineering and scientific computing; however, its pre-processing, solver configuration, and post-processing stages are typically time-consuming and demand extensive specialized knowledge. This paper introduces an automated simulation system, termed MooseAgent, specifically for the multiphysics simulation framework Moose. The system, leveraging Large Language Model (LLM) and multi-agent technology, aims to interpret user simulation requirements expressed in natural language and automatically generate Moose input files by employing task decomposition and multi-round iterative verification strategies. To enhance generation accuracy and mitigate model hallucinations, the system constructs and utilizes a dedicated vector database comprising annotated Moose input cards and function documentation. Experimental evaluations across typical cases, including heat transfer, mechanics, phase-field, and multiphysics coupling, demonstrate that MooseAgent can automate the Moose simulation process to a considerable extent, achieving an average success rate of 78% across all tasks. Furthermore, the system exhibits significant cost-effectiveness, with an average expenditure of 0.75 RMB per case. These findings underscore the transformative potential of LLM in automating simulation analysis.

1. Introduction

The Finite Element Method (FEM), as a powerful numerical computation tool, plays a core role in solving multiphysics problems such as structural mechanics, heat transfer, electromagnetic field analysis, and fluid-structure interaction [1, 2]. Among these, Moose [1], with its highly modular and flexible multiphysics coupling capabilities, has found widespread application in the nuclear energy. However, the traditional FEM simulation workflow is often complex, requiring significant time and effort from preprocessing steps like geometry creation and mesh generation, to solver parameter configuration, and post-processing stages such as results visualization and analysis. This process also demands a high level of expertise from the operator. Improper configuration at any stage can lead to simulation failure or distorted results, which undoubtedly creates a significant technical barrier, to some extent limiting the broader application and popularization of the FEM and the Moose framework.

In recent years, the field of artificial intelligence (AI) has witnessed remarkable advancements, particularly in deep learning (DL) technologies, which have achieved unprecedented breakthroughs. The integration of AI with scientific research holds the promise of accelerating simulation and design processes across various scientific and engineering domains. In the field of nuclear energy, AI is commonly applied to modeling [3] or expediting scientific computations [4, 5], and recent developments in large language model (LLM) technology have also shown significant potential for scientific research applications.

Large Language Models (LLMs) have shown immense potential in natural language understanding, knowledge reasoning, and task automation, bringing new opportunities to revolutionize scientific computing workflows [6, 7, 8, 9]. Combining LLMs with Multi-Agent Systems (MAS) to build intelligent software agents capable of collaboratively completing complex tasks has become a research hotspot. Based on their level of encapsulation and automation, existing MAS can be broadly categorized: the first category, such as LangChain [10] and LangGraph [11], provides basic building blocks, requiring developers to code and manage agent interactions themselves, offering high flexibility but at a correspondingly higher development cost. The second category, including CrewAI [12], AutoGen [13], and MetaGPT [14], offers a higher level of encapsulation where users define task objectives and agent roles, and the framework manages inter-agent communication and collaboration. The third category further lowers the barrier to entry through graphical user interfaces (e.g., Dify [15], Bisheng [16]), supporting no-code workflow construction. More advanced research explores treating the multi-agent workflow itself as an optimizable object that can autonomously learn and adjust its collaboration strategies [17, 18]. The application of LLM-driven multi-agent systems in scientific simulation and software automation has already shown initial success. For instance, in computational fluid dynamics, researchers have experimented with using multi-agent systems to assist software like OpenFOAM, achieving an automated chain from natural language requirements to simulation execution [19, 20]. The AutoFLUKA [21] project utilizes LangChain to automate Monte Carlo simulations in FLUKA. These advancements fully demonstrate the great potential of LLM-driven multi-agent systems in enhancing user experience and automation levels for complex engineering software. Despite the broad application prospects of LLMs and multi-agent

*Corresponding author

**Corresponding author

✉ taozhan22@gmail.com (T. Zhang); 1575732528@qq.com (Y. Jiao)

systems in numerous fields, related automation research in the finite element analysis domain, especially for highly complex multiphysics coupling simulation platforms like Moose, remains relatively scarce. The Moose framework offers a higher degree of freedom and more complex configuration than other finite element software, making it more difficult to master. This positions it as an ideal application scenario for introducing intelligent automation assistance, with the potential to empower more non-expert users to conveniently solve practical engineering problems using Moose.

Based on the aforementioned background and practical needs, this paper proposes an open-source automated Moose simulation multi-agent system named MooseAgent. MooseAgent employs a task decomposition strategy, combined with Retrieval-Augmented Generation (RAG) using a vector knowledge base that contains a large number of Moose input files and function documentation, and supplemented by a multi-round iterative validation and correction mechanism to generate correct Moose input files and execute calculations. In addition to significantly improving the efficiency of simulation design with Moose, MooseAgent also lays the technical foundation for developing more advanced fuel or reactor design intelligent agents in the future. This paper will systematically elaborate on the architectural design of MooseAgent, its core algorithmic workflow, the construction details of its specialized knowledge base, and will comprehensively evaluate its performance through a series of typical simulation cases. The code for the proposed MooseAgent is open-source and available at <https://github.com/taozhan18/MooseAgent>.

2. Method

2.1. Overall architecture

The overall framework of MooseAgent is shown in Fig. 1. Its core idea is to leverage the natural language understanding capabilities of LLMs and the collaborative problem-solving abilities of MAS to achieve an automated workflow from user requirements to Moose simulation results. The entire framework is built on LangGraph, and we assign specific roles to each agent and precisely define their interaction sequences, collaborative logic, and the information passed between them. This ensures efficient task execution and robust error handling.

The framework can be divided into three primary parts, each with specific responsibilities and actions:

Alignment: This part generates a detailed simulation task description based on the user's requirements and aligns the simulation goals through interaction with the user.

Write Input Card: This part asynchronously writes the MOOSE input file based on the detailed simulation task description.

Execute-Analyze-Correct: This part corrects errors in the input file through multiple rounds of iteration to ensure it runs correctly.

Each core function is executed by a dedicated agent. Every agent operates based on a unique set of prompts, which

consists of a 'System Prompt' and a 'User Input Prompt'. The prompt design for the alignment agent is illustrated in Fig. 2, while the complete code for all agent prompts is available in our repository.

First, the user inputs the simulation requirements in natural language. The agent then begins by deeply understanding and parsing these requirements, which may be vague or unclear. It determines the number of Moose input files needed and the specific tasks each file must accomplish. During this process, the LLM may engage in a multi-turn dialogue with the user to clarify any ambiguous descriptions, thereby ensuring the accuracy of the task definition. Subsequently, the system asynchronously writes the Moose input files based on the task descriptions. For tasks requiring multiple input files, asynchronous execution enhances the efficiency of code generation. In the writing process, the system first formulates a search query based on the simulation task description. It then retrieves similar input files from a simulation case database to serve as references for generating the initial input file code.

Once an input file has been written, a multi-round "execute-analyze-correct" iterative process begins. If an error or warning is encountered during the Moose execution, the error information is returned. The agent first analyzes the error history to determine if it is encountering a recurring error. If so, it regenerates the input file and suggests a different implementation approach to avoid the error. Otherwise, it calls upon a "Moose Assistant" for a specific solution. The "Moose Assistant" is an LLM configured with a Moose knowledge base, enabling it to selectively retrieve relevant content to answer technical questions about the Moose simulation. This iterative process continues until the simulation runs successfully or a preset maximum number of iterations is reached.

2.2. database

To reduce the "hallucination" phenomenon in LLMs—where the model may generate inaccurate or irrelevant content—and enhance the accuracy of generated content, this paper constructs a specialized Moose vector database composed of two core parts.

The first part consists of annotated Moose input files. Over 8,000 input file instances were collected from Moose's official code repository, primarily sourced from various test cases and tutorials. However, the original input files largely lack detailed comments. Using them directly for retrieval could make it difficult for the model to understand their true intent or lead to misuse. To address this, an automated input file annotation workflow was designed and implemented, as shown in Fig. 4. This workflow involves the following steps:

- First, it randomly selects samples from the unannotated input files.
- Next, by analyzing the types of Moose functions used, it retrieves descriptions of their functionality, usage methods, and relevant parameter details from the Moose documentation.

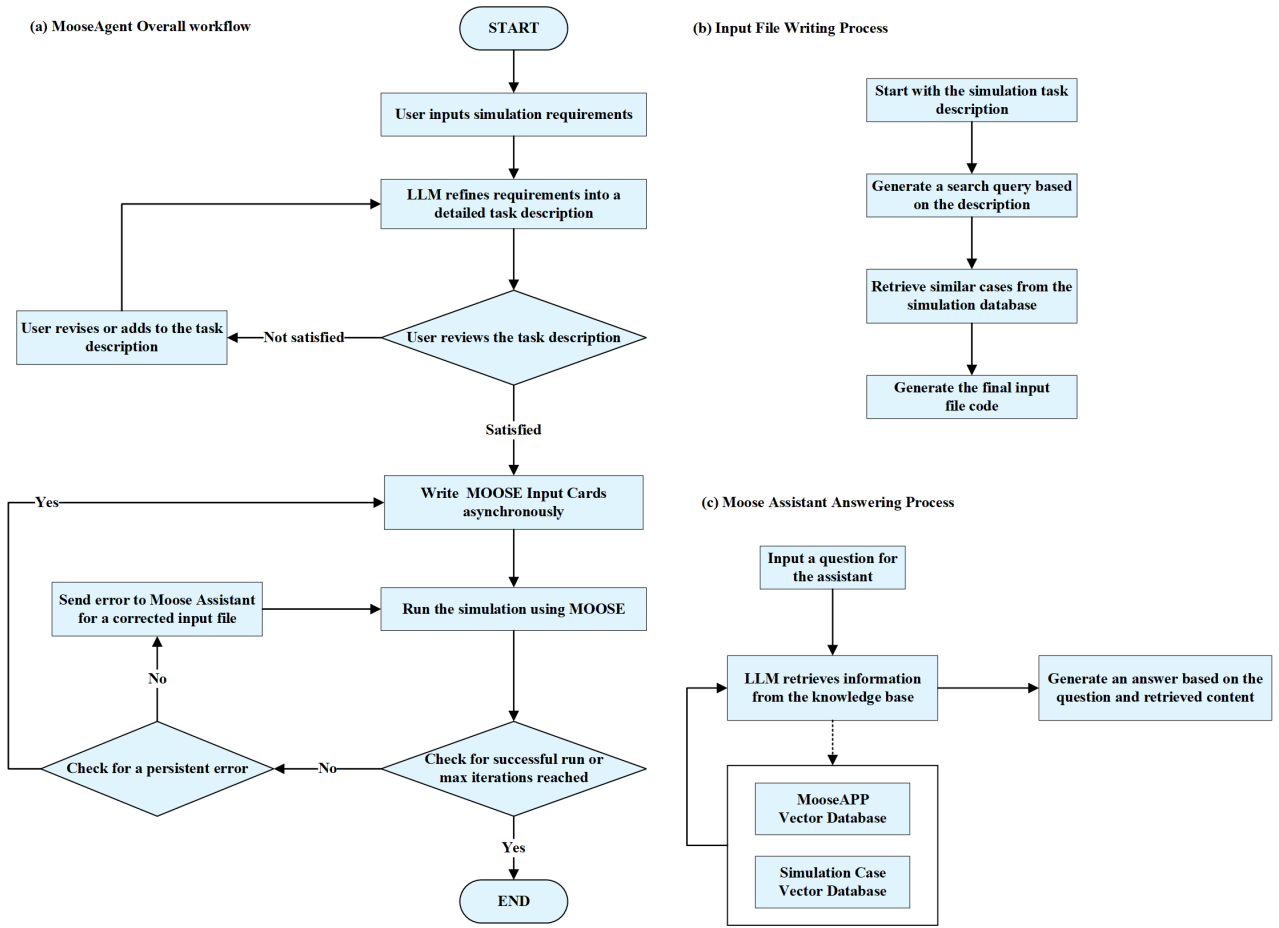


Figure 1: Overview of MooseAgent.

- Finally, using RAG technology, it leverages the retrieved contextual information to drive an LLM to generate detailed comments for the input file. This annotation process is executed iteratively, continuously expanding the library of annotated input files. All annotated input files are stored in JSON format, with each entry containing the input file's name, a summary of its simulation task, and the input file code with detailed comments.

The second part is the detailed documentation for all functions included in Moose. While Moose has a powerful documentation system that can export the basic usage of all functions via commands, it often lacks descriptions of specific functionalities and application scenarios. To address this, a Python script was developed to scan the Moose documentation library, locating and extracting detailed textual descriptions and usage explanations for each function and its core features. The extracted information is organized and stored in JSON format, with each entry containing the function's name, a comprehensive description of its functionality, and detailed explanations of its input parameters, including data types and default values. Fig.3 shows an example of the database.

Together, these two sources constitute a comprehensive knowledge base that serves as the foundation for MooseAgent. To make this information accessible, we employ the state-of-the-art BGE-M3 embedding model [22] to encode the entire corpus of textual data into semantically rich, high-dimensional vectors. These vectors are then stored and indexed in a FAISS vector database [23], a system chosen for its exceptional efficiency and ease of use. FAISS is particularly well-suited for fast similarity searches and indexing of dense vectors. For any given query, the system calculates relevance using a similarity score and selects the top 3 chunks with the highest scores. The retrieved information is then combined with the user's original message using a simple stacking method, which creates an enriched context. This method involves layering the retrieved chunks directly with the user's query to provide a comprehensive context for the LLM. Through the above technologies, we effectively provide the LLM with accurate domain knowledge, thereby significantly enhancing the accuracy, reliability, and overall success rate of the entire automated workflow.

- **Phase Change Heat Conduction:** Simulates the movement of the phase change front during the melting/solidification process of a one-dimensional metal rod driven by a temperature difference at its ends.
- **Phase Field:** Uses a phase-field model to simulate the solidification process of a two-dimensional pure metal driven by low temperature, observing the formation, growth, and interface evolution of the solid phase.
- **Prismatic Fuel:** Loads a predefined 16-sided prismatic fuel assembly mesh, sets temperature and mechanical boundary conditions, and solves for the fuel's thermal and mechanical performance.
- **Thermal-Mechanical Coupling:** Utilizes Moose's Multiapp functionality to simulate the thermal-structural coupled behavior of a two-dimensional rectangular thin plate, analyzing thermal expansion and displacement.

3.2. results

Table 2 summarizes the performance of MooseAgent across 9 different test cases, achieving an average success rate of 78% over all cases. MooseAgent attained a success rate of 80% or higher in handling cases of steady-state and transient heat conduction, linear elasticity, phase-field, and prismatic fuel. This indicates that for problems where the physical models are relatively mature, the solution process is fairly standard, and the knowledge base is likely to contain a greater number of similar reference cases, MooseAgent can accurately understand user requirements and generate correct input files. For problems involving plasticity, phase change heat conduction, and porous media flow, the success rate was 60% for each. This reflects that MooseAgent's capability is still adequate for handling slightly more complex problems, such as those involving non-linear material behavior, physical processes with phase changes, and fluid dynamics. The thermal-mechanical coupling case had the lowest success rate at only 40%. This is because multiphysics coupling problems are inherently difficult to configure, involving complex interactions and data transfer between multiple physical domains, which poses a higher challenge to the agent's comprehensive understanding, task decomposition, and collaborative processing abilities. In terms of token usage, there were significant differences between cases, mainly related to task difficulty. However, the token count for all cases did not exceed 100,000, with an average consumption of 47,017 tokens. If calculated based on the pricing for calling the DeepSeek-R1 model, this would cost approximately 0.75 RMB. Therefore, MooseAgent can perform simulation tasks at a low cost.

Fig. 5 illustrates a typical MooseAgent workflow using a prismatic fuel element simulation as an example. Initially, the user describes the basic requirements for the thermal-mechanical simulation in natural language, specifying details such as the names of boundaries and blocks in the mesh file and their corresponding boundary conditions. LLM then comprehends and refines these requirements, generating a

Table 1
Performance of MooseAgent

Case	Success Rate	Tokens
Steady-State Heat Conduction	1	24673
Transient Heat Conduction	1	26729
Linear Elasticity	1	40857
Plasticity	0.6	15874
Phase Change Heat Conduction	1	64592
Porous Media Flow	1	104766
Phase Field	1	86696
Prismatic Fuel	1	37424
Thermal-Mechanical Coupling	0.8	79075

detailed description that encompasses the mesh, physical processes, boundary conditions, material properties, and solver settings. This description is displayed on the console for user confirmation. Once the requirements are confirmed, a complete input file is generated and executed. If MOOSE returns an error during execution, a multi-turn iterative correction process is initiated. In this process, the LLM analyzes the error message and source code to identify the cause, then modifies the input file for re-execution. Ultimately, the simulation runs successfully, yielding contour plots that show the distribution of the fuel's thermal-mechanical performance.

An analysis of failed runs reveals that all unsuccessful attempts were caught in an infinite loop. Specifically, the agent would get stuck repeatedly trying to fix an error by calling a particular function. Because it could never successfully execute the function call, it would eventually exit after reaching the maximum number of iterations. The root cause of this issue is twofold. First, as an open-source software, Moose has a vast number of available functions, making it difficult for LLM to select the most appropriate one for a given task. Additionally, LLM lacks accuracy in handling some of Moose's fundamental functionalities. Specifically, when writing function expressions, understanding the basic format of input cards, defining constants, and establishing relationships between different modules, it occasionally falls into a loop of irreparable errors. To address this, future improvements could focus on two key areas. One approach is to incorporate a human feedback mechanism to guide the model on how to escape its current error loops. Another strategy is to fine-tune the large model to enhance its understanding of specific Moose conventions. Both methods are expected to increase the overall task success rate.

3.3. Ablation Analysis

4. Conclusion

This paper proposes an automated Moose simulation multi-agent system named MooseAgent. The system can understand user requirements in natural language and combines task decomposition, Retrieval-Augmented Generation, and a multi-round iterative validation and correction strategy to automatically generate Moose input files and perform simulations, thereby lowering the barrier to entry for Moose and improving simulation efficiency. Experimental results

User:

Steady-state thermodynamic simulations are performed for 16 prismatic fuel elements, using the assumption of plane strain. (detail setting)

Alignment:

Based on the user's simulation requirements, only one input file is needed to complete the steady-state thermodynamic simulation for the 16 prismatic fuel elements. The simulation involves thermal and mechanical coupling but does not require multi-physics coupling with separate input files. Here is the detailed description:

File_name1: prismatic_fuel_thermomech.i

Description:

- **Geometry/Mesh**: (detail information).
- **Physical Process**: The simulation involves steady-state thermal conduction and linear elasticity (thermo-mechanical coupling).
- **Boundary Conditions**:
 - (detail information)
- **Material Properties**:
 - (detail information)
- **Solution Setting**:
 - (detail information)
- **Post Processing**: Output temperature, displacement, and stress fields for visualization and analysis.

Architect:

Write the input card to file.

Runner:

*** ERROR ***

Task init_mesh is not registered to build MeshGenerator derived objects
(detail information)

Modify:

*** ERROR ***

The error in file: prismatic_fuel_thermomech.i. The reason is that: *** ERROR ***

Task init_mesh is not registered to build MeshGenerator derived objects

The error indicates that the FileMeshGenerator is not being properly initialized or registered in the MOOSE framework. (detail information).

To fix this, (detail information).

Rewrite the input card to file.

...

Finish:

SUCCESS:

(execution information)

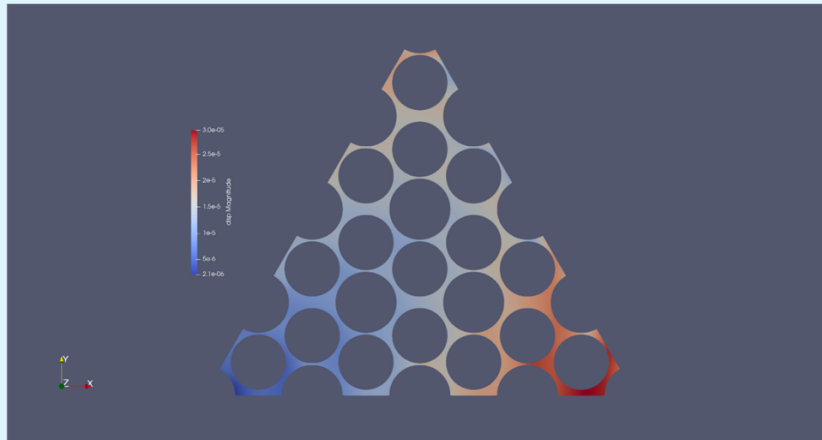


Figure 5: Prismatic fuel element.

Table 2

Performance of MooseAgent without database

Case	Success Rate	Tokens
Steady-State Heat Conduction	1	11194
Transient Heat Conduction	1	33871
Linear Elasticity	0.8	22445
Plasticity	0.2	100590
Phase Change Heat Conduction	0.6	35241
Porous Media Flow	0.8	77502
Phase Field	0.8	97993
Prismatic Fuel	1	
Thermal-Mechanical Coupling	0.6	94468

show that MooseAgent achieves a high accuracy rate on problems with relatively simple physical processes and can also succeed on more complex problems through multiple attempts, with an average success rate of 78% across all cases. Furthermore, the token consumption for all cases is kept at a low level, with the average cost of running a single simulation task being approximately 0.75 RMB, demonstrating its high cost-effectiveness. Future work will focus on optimizing knowledge retrieval and iterative strategies, and will consider introducing human feedback into the iteration process to further enhance the system's performance on complex problems.

CRedit authorship contribution statement

Tao Zhang: Investigation, Methodology, Software, Visualization, Writing - original draft. **Zhenhai Liu:** Conceptualization, Software, Validation, Writing - review & editing. **Guangliang Yang:** Methodology. **Yong Xin:** Project administration. **Yuanming Li:** Project administration. **Yongjun Jiao:** Conceptualization, Resources, Supervision, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code for MooseAgent has been made open source and is available at <https://github.com/taozhan18/MooseAgent>.

Acknowledgements

This study did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1] C. Icenhour, S. Keniley, C. DeChant, C. Permann, A. Lindsay, R. Martineau, D. Curreli, S. Shannon, Multi-physics object oriented simulation environment (moose), Technical report, Idaho National Lab.(INL), Idaho Falls, ID (United States) (2018).
- [2] P. Kohnke, Ansys, in: Finite element systems: a handbook, pages 19–25, Springer (1982).
- [3] T. Zhang, Y. Jiao, Z. Liu, S. Xing, H. Wang, K. Zhang, Y. Li, Modeling of zirconium alloy cladding corrosion behavior based on neural ordinary differential equation, Nuclear Engineering and Technology, 57 (2025) 103251.
- [4] T. Zhang, Y. Jiao, Z. Liu, Y. Li, H. Wang, Learning to simulate quasi-steady-state fuel rod performance with hybrid Fourier neural operator, Progress in Nuclear Energy, 186 (2025) 105769.
- [5] T. Zhang, Z. Liu, F. Qi, Y. Jiao, T. Wu, M2PDE: Compositional Generative Multiphysics and Multi-component PDE Simulation, in: Forty-second International Conference on Machine Learning (2025).
- [6] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, D. Ha, The ai scientist: Towards fully automated open-ended scientific discovery, arXiv preprint arXiv:2408.06292, (2024).
- [7] D. A. Boiko, R. MacKnight, B. Kline, G. Gomes, Autonomous chemical research with large language models, Nature, 624 (2023) 570–578.
- [8] S. Yang, S. Batzner, R. Gao, M. Aykol, A. Gaunt, B. C. McMorrow, D. Jimenez Rezende, D. Schuurmans, I. Mordatch, E. D. Cubuk, Generative hierarchical materials search, Advances in Neural Information Processing Systems, 37 (2024) 38799–38819.
- [9] Y. Tang, W. Xu, J. Cao, J. Ma, W. Gao, S. Farrell, B. Erichson, M. W. Mahoney, A. Nonaka, Z. Yao, MatterChat: A Multi-Modal LLM for Material Science, arXiv preprint arXiv:2502.13107, (2025).
- [10] O. Topsakal, T. C. Akinci, Creating large language model applications utilizing langchain: A primer on developing llm apps fast, in: International Conference on Applied Engineering and Natural Sciences, volume 1 (2023) pages 1050–1056.
- [11] H. Chen, Y. Ding, Implementing traffic agent based on LangGraph, in: Fourth International Conference on Intelligent Traffic Systems and Smart City (ITSSC 2024), volume 13422, SPIE (2025) pages 582–587.
- [12] P. Venkadesh, S. Divya, K. S. Kumar, Unlocking ai creativity: A multi-agent approach with crewai, Journal of Trends in Computer Science Smart Technology, 6 (2024) 338–356.
- [13] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, et al., Autogen: Enabling next-gen llm applications via multi-agent conversation, arXiv preprint arXiv:2308.08155, (2023).
- [14] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, et al., Metagpt: Meta programming for multi-agent collaborative framework, arXiv preprint arXiv:2308.00352, 3 (2023) 6.
- [15] langgenius, dify, <https://github.com/langgenius/dify> (2021), [2024-07-25].
- [16] DataElem, bisheng, <https://github.com/dataelement/bisheng> (2025), [2025-07-05].
- [17] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, P. Lu, Z. Huang, C. Guestrin, J. Zou, Optimizing generative AI by backpropagating language model feedback, Nature, 639 (2025) 609–616.
- [18] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhu, X. Cheng, S. Hong, J. Wang, et al., Afrow: Automating agentic workflow generation, arXiv preprint arXiv:2410.10762, (2024).
- [19] Y. Chen, X. Zhu, H. Zhou, Z. Ren, MetaOpenFOAM 2.0: Large Language Model Driven Chain of Thought for Automating CFD Simulation and Post-Processing, arXiv preprint arXiv:2502.00498, (2025).
- [20] S. Pandey, R. Xu, W. Wang, X. Chu, OpenFOAMGPT: a RAG-Augmented LLM Agent for OpenFOAM-Based Computational Fluid Dynamics, arXiv preprint arXiv:2501.06327, (2025).
- [21] Z. N. Ndum, J. Tao, J. Ford, Y. Liu, AutoFLUKA: A Large Language Model Based Framework for Automating Monte Carlo Simulations in FLUKA, arXiv preprint arXiv:2410.15222, (2024).
- [22] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, arXiv preprint

arXiv:2402.03216, (2024).

- [23] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, H. Jégou, The faiss library, arXiv preprint arXiv:2401.08281, (2024).